

Comparison Operators

Operator	Meaning
==	Equal to
!=	Not equal to
<	Less than
>	Greater Than
<=	Less than or Equal to
>=	Greater than or Equal to

Boolean Operators

There are three Boolean operators: and, or, and not

The and Operator's *Truth* Table:

Expression	Evaluates to
True and True	True
True and False	False
False and True	False
False and False	False

The or Operator's *Truth* Table:

Expression	Evaluates to
True or True	True
True or False	True
False or True	True
False or False	False

The not Operator's *Truth* Table:

Expression	Evaluates to
not True	False
not False	True

Mixing Operators-----

Indentation

In some languages, especially Python, indentation is *syntactically significant* — it actually determines which lines belong to which block. The interpreter uses it to know what's inside a loop, function, if-statement, etc.

if Statements

The if statement evaluates an expression, and if that expression is True, it then executes the following indented code:

```
name = 'Debora'
```

```
if name != 'George':
```

```
    print('You are not George')
```

The else statement executes only if the evaluation of the if and all the elif expressions are False

if-else: execute different code based on condition

```
name = 'Debora'
```

```
if name == 'George':
```

```
    print('Hi, George.')
```

```
else: # Execute if condition is False
```

```
    print('You are not George')
```

Only after the if statement expression is False, the elif statement is evaluated and executed:

if-elif: check multiple conditions in sequence

```
name = 'George'
```

```
if name == 'Debora':
```

```
    print('Hi Debora!')
```

```
elif name == 'George': # Checked only if previous condition is False
```

```
    print('Hi George!')
```

```
ex—
```

```
name = 'Antony'
```

```
if name == 'Debora':
```

```
    print('Hi Debora!')
```

```
elif name == 'George':
```

```
    print('Hi George!')
```

```
else:
```

```
    print('Who are you?')
```

Ternary Conditional Operator

it offers one-line code to evaluate the first expression if the condition is true, and otherwise it evaluates the second expression.

<expression1> if <condition> else <expression2>

Ternary operator: one-line conditional expression

Syntax: value_if_true if condition else value_if_false

```
print('kid' if age < 18 else 'adult')
```

```
age = 15
```

this ternary operator:

```
print('kid' if age < 13 else 'teen' if age < 18 else 'adult')
```

Switch-Case Statement-----

```
response_code = 201
```

```
match response_code:
```

```
    case 200:
```

```
        print("OK")
```

```
    case 201:
```

```
        print("Created")
```

```
    case 300:
```

```
        print("Multiple Choices")
```

```
    case 307:
```

```
        print("Temporary Redirect")
```

```
    case 404:
```

```
        print("404 Not Found")
```

```
    case 500:
```

```
        print("Internal Server Error")
```

case 502:

```
print("502 Bad Gateway")
```

ex---

```
response_code = "300"
```

```
match response_code:
```

```
    case int():
```

```
        print('Code is a number')
```

```
    case str():
```

```
        print('Code is a string')
```

```
    case _:
```

```
        print('Code is neither a string nor a number')
```

For loop-----

The for loop iterates over a list, tuple, dictionary, set or string:

for loop: iterate over each item in a sequence

```
pets = ['Bella', 'Milo', 'Loki']
```

```
for pet in pets: # Loop through each pet in the list
```

```
    print(pet) # Print each pet name
```

while Loop Statements-----

The while statement is used for repeated execution as long as an expression is True:

while loop: repeat code as long as condition is True

```
spam = 0
```

```
while spam < 5: # Continue while spam is less than 5
```

```
    print('Hello, world.')
```

```
    spam = spam + 1 # Increment counter to avoid infinite loop
```

break Statements----

If the execution reaches a break statement, it immediately exits the while loop's clause:

break statement: exit loop immediately when encountered

while True: # Infinite loop

name = input('Please type your name: ')

if name == 'your name':

break # Exit the loop immediately

print('Thank you!')

continue Statements----

When the program execution reaches a continue statement, the program execution immediately jumps back to the start of the loop.

continue statement: skip rest of loop iteration and start next iteration

while True:

name = input('Who are you? ')

if name != 'Joe':

continue # Skip to next iteration, don't ask for password

password = input('Password? (It is a fish.): ')

if password == 'swordfish':

break # Exit loop when password is correct

print('Access granted.')

The range() function -----

The range() function returns a sequence of numbers. It starts from 0, increments by 1, and stops before a specified number:

```
for i in range(5):
```

```
    print(f'Will stop at 5! or 4? ({i})')
```

The range() function can also modify its 3 default arguments. The first two will be the start and stop values, and the third will be the step argument. The step is the amount that the variable is increased by after each iteration.

```
# range(start, stop, step)
```

```
for i in range(0, 10, 2):
```

```
    print(i)
```

```
for i in range(5, -1, -1):
```

```
    print(i)
```

Python Functions-----

A function can take arguments and return values:

In the following example, the function say_hello receives the argument "name" and prints a greeting:

```
# Define a function that takes one argument
```

```
def say_hello(name):
```

```
    print(f'Hello {name}')
```

```
# Call the function with a string argument
```

```
say_hello('Carlos')
```

To improve code readability, we should be as explicit as possible. We can achieve this in our functions by using Keyword Arguments:

Function with multiple parameters

```
def say_hi(name, greeting):  
    print(f"{greeting} {name}")
```

Positional arguments: order matters

```
say_hi('John', 'Hello')
```

Return Values-----

When creating a function using the def statement, you can specify what the return value should be with a return statement. A return statement consists of the following:

- The return keyword.
- The value or expression that the function should return.

Function that returns a value using return statement

```
def sum_two_numbers(number_1, number_2):  
    return number_1 + number_2
```

Call function and store the returned value

```
result = sum_two_numbers(7, 8)  
print(result)
```

Function that returns a value using return statement

```
def sum_two_numbers(number_1, number_2):  
    return number_1 + number_2
```

Call function and store the returned value

```
result = sum_two_numbers(7, 8)  
  
print(result)
```

Local and Global Scope-----

- Code in the global scope cannot use any local variables.
- However, a local scope can access global variables.
- Code in a function's local scope cannot use variables in any other local scope.
- You can use the same name for different variables if they are in different scopes. That is, there can be a local variable named spam and a global variable also named spam

Global variable: accessible everywhere

```
global_variable = 'I am available everywhere'
```

```
def some_function():
```

```
    print(global_variable) # Can access global variable
```

```
    # Local variable: only exists within this function
```

```
    local_variable = "only available within this function"
```

```
    print(local_variable)
```

This will raise NameError: local_variable doesn't exist in global scope

```
print(local_variable)
```

The global Statement-----

If you need to modify a global variable from within a function, use the global statement:

Use 'global' keyword to modify global variable from inside function

```
def spam():
```

```
    global eggs # Declare that we're modifying the global variable
```

```
    eggs = 'spam' # This changes the global variable
```

```
eggs = 'global'
```

```
spam() # Function modifies global variable
```

```
print(eggs) # Prints 'spam', not 'global'
```

There are four rules to tell whether a variable is in a local scope or global scope:

1. If a variable is being used in the global scope (that is, outside all functions), then it is always a global variable.
2. If there is a global statement for that variable in a function, it is a global variable.
3. Otherwise, if the variable is used in an assignment statement in the function, it is a local variable.
4. But if the variable is not used in an assignment statement, it is a global variable.

Lambda Functions-----

In Python, a lambda function is a single-line, anonymous function, which can have any number of arguments, but it can only have one expression.

Regular function definition

```
def add(x, y):  
    return x + y
```

```
add(5, 3)
```

Is equivalent to the *lambda* function:

Lambda function: anonymous function defined in one line

Syntax: lambda arguments: expression

```
add = lambda x, y: x + y
```

```
add(5, 3)
```

Reading and Writing Files-----

To read/write to a file in Python, you will want to use the with statement, which will close the file for you after you are done, managing the available resources for you.

Opening and reading files---

The open function opens a file and return a corresponding file object.

Read file using 'with' statement: automatically closes file when done

```
with open('/home/labex/project/hi.txt') as hello_file:
```

```
    hello_content = hello_file.read() # Read entire file content
```

```
hello_content
```

Writing to files-----

Write to file: 'w' mode overwrites existing file

with open('bacon.txt', 'w') as bacon_file: # 'w' = write mode

bacon_file.write('Hello world!\n') # Returns number of characters written

Append to file: 'a' mode appends to existing file

with open('bacon.txt', 'a') as bacon_file: # 'a' = append mode

bacon_file.write('Bacon is not a vegetable.')

Python Exception Handling-----

Python has many built-in exceptions that are raised when a program encounters an error, and most external libraries, like the popular Requests, include his own custom exceptions that we will need to deal to.

Basic exception handling

You can't divide by zero, that is a mathematical true, and if you try to do it in Python, the interpreter will raise the built-in exception ZeroDivisionError:

```
def divide(dividend , divisor):
```

```
    print(dividend / divisor)
```

```
divide(dividend=10, divisor=5)
```

Let's say we don't want our program to stop its execution or show the user an output he will not understand. Say we want to print a useful and clear message, then we need to *handle* the exception with the try and except keywords:

try-except: handle exceptions gracefully

```
def divide(dividend , divisor):
```

```
    try: # Try to execute this code
```

```
print(dividend / divisor)
```

```
except ZeroDivisionError: # Catch specific exception type
```

```
print('You can not divide by 0')
```

```
divide(dividend=10, divisor=5)
```

```
divide(dividend=10, divisor=0)
```

Handling Multiple exceptions using one exception block-----

```
# Handle multiple exceptions in one except block
```

```
def divide(dividend , divisor):
```

```
    try:
```

```
        if (dividend == 10):
```

```
            var = 'str' + 1 # This will raise TypeError
```

```
        else:
```

```
            print(dividend / divisor)
```

```
    except (ZeroDivisionError, TypeError) as error: # Catch multiple exception types
```

```
        print(error) # Print the error message
```

```
divide(dividend=20, divisor=5)
```

Finally code in exception handling----

```
# finally block: always executes regardless of exceptions
```

```
def divide(dividend , divisor):
```

```
    try:
```

```
        print(dividend / divisor)
```

```
    except ZeroDivisionError:
```

```
        print('You can not divide by 0')
```

```
    finally: # Always executes, even if exception occurs
```

```
        print('Execution finished')
```

```
divide(dividend=10, divisor=5)
```

1. FizzBuzz — Print numbers 1 to 100. For multiples of 3 print "Fizz", multiples of 5 print "Buzz", multiples of both 3 and 5 print "FizzBuzz".

2. Grade Calculator — Given a numeric score (0–100), return a letter grade (A/B/C/D/F) using if/else chains.

3. Leap Year Checker — Determine if a given year is a leap year (divisible by 4, but not by 100 unless also by 400).

4. Triangle Type — Given three side lengths, determine if the triangle is equilateral, isosceles, scalene, or invalid (doesn't satisfy triangle inequality).

5. Simple Calculator — Take two numbers and an operator (+, -, *, /) as input; use a switch statement to perform the operation, handling division by zero.

6. Day Name from Number — Given an integer 1–7, return the day of the week using switch/case, with a default case for invalid input.

7. BMI Category — Compute BMI from height/weight, then classify as Underweight/Normal/Overweight/Obese using nested conditionals.

8. Sum and Average — Loop through an array of numbers and compute their sum and average.

9. Reverse a Number — Given an integer like 12345, reverse its digits using a while loop (no string conversion allowed).

10. Prime Checker — Write a function that loops to check if a number is prime; then loop through 1–100 and print all primes.

11. Multiplication Table — Using nested loops, print a multiplication table (e.g., 1–10 × 1–10) in grid format.

12. Pattern Printing — Use nested loops to print a right triangle of stars, then a pyramid pattern:

*

**

13. Palindrome Checker — Use a loop to check if a string or number reads the same forwards and backwards.

14. Collatz Sequence — Starting from any positive integer, loop until you reach 1, applying: if even, divide by 2; if odd, multiply by 3 and add 1. Count the steps.

Functions (basic, recursive, higher-order)

15. Factorial (Two Ways) — Write a factorial function twice: once iteratively (loop) and once recursively. Compare the two approaches.

16. Fibonacci Sequence — Write a function that returns the nth Fibonacci number, first recursively, then with memoization for efficiency.

17. Array Filter/Map (Higher-Order) — Write a custom filterEvens(array) and mapDouble(array) function without using built-in filter/map, to understand what they do internally.

18. Default Parameters & Overload-like Behavior — Write a greet(name, greeting = "Hello") function that works whether or not a greeting is provided.

19. Recursive String Reversal — Write a function that reverses a string using recursion (no built-in reverse methods).

20. Combine Everything: Simple Calculator App — Write a function `calculate(a, b, operator)` that uses control flow to select the operation, wrap it in a loop that keeps asking the user for new calculations until they type "exit," and handle invalid input gracefully with functions for validation.



Want me to also provide solutions (in a specific language like Python, JavaScript, or Java), or turn this into a printable practice sheet?

20. Combine Everything: Simple Calculator App — Write a function `calculate(a, b, operator)` that uses control flow to select the operation, wrap it in a loop that keeps asking the user for new calculations until they type "exit," and handle invalid input gracefully with functions for validation.

Lunetron Web Services